Mini Project Report

of

Database Systems Lab (CSE 2212)

College Quiz Management System

SUBMITTED BY

Name	Reg. No.	Roll No.
Ankitha M	240953194	16
Arnav Varshney	240953382	31
Shreeniketh E R	240953148	10

Section: CCE-B

School of Computer Engineering
Manipal Institute of Technology, Manipal.

April 2026

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Manipal Institute of Technology, Manipal

April 2026

CERTIFICATE

This is to certify that the project titled **College Quiz Management System** is a record of the bonafide work done by **Ankitha M (Reg. No. 240953194), Arnav Varshney (Reg. No. 240953382), and Shreeniketh E R (Reg. No. 240953148)**, submitted in partial fulfilment of the requirements for the award of the Degree of Bachelor of Technology (B.Tech.) in **Computer and Communication Engineering** of Manipal Institute of Technology, Manipal, Karnataka, (A Constituent Institute of Manipal Academy of Higher Education), during the academic year 2025-2026.

Name and Signature of Examiners:

1. Dr. Chitra K, Assistant Professor, SCE.
2. Dr. Anita S Kini, Assistant Professor, SCE.

TABLE OF CONTENTS

Abstract 1
Chapter 1: Introduction 2
Chapter 2: Problem Statement & Objectives 3
Chapter 3: Methodology 4
Chapter 4: ER Diagram & Relational Tables 5
Chapter 5: DDL Commands & SQL Procedures 7
Chapter 6: Results & Snapshots 10
Chapter 7: Conclusion, Limitations & Future Work 12
Chapter 8: References 13

ABSTRACT

The College Quiz Management System is a web-based application designed to streamline the process of conducting, managing, and evaluating quizzes in educational institutions. Traditional paper-based assessments are time-consuming and prone to human error, while many existing online solutions lack adequate security and usability features.

This project provides a secure, role-based platform accessible on any device via a web browser. The system enables teachers to create quizzes with multiple-choice and true/false questions, manage subjects, control result publication, and monitor student performance. Students can register, self-enroll in subjects, and take time-bound quizzes with automatic grading.

The application is built using Python Flask for the backend, MySQL for relational database management, and Jinja2 HTML templates with CSS for the frontend. Key features include automated grading, time-bound assessments with a JavaScript countdown timer, enrollment-based quiz access, question order randomization, and controlled result visibility.

CHAPTER 1: INTRODUCTION

1.1 Background

Educational institutions continuously seek efficient methods for conducting assessments and evaluating student performance. The conventional approach of paper-based quizzes presents several challenges including manual grading, delayed results, difficulty in tracking performance trends, and significant administrative overhead.

With the rapid adoption of digital technologies in education, web-based assessment systems have emerged as a practical solution. These systems can automate grading, provide instant feedback, and maintain comprehensive records of student performance over time.

1.2 Project Overview

The College Quiz Management System is a full-stack web application developed as part of the Database Systems Lab (CSE 2212) course at Manipal Institute of Technology. The system provides a centralized platform for teachers and students to interact through a structured quiz workflow.

The application supports two distinct user roles — Teacher and Student — each with a dedicated set of features accessible through a single unified login portal with automatic role detection.

1.3 Technology Stack

Component	Technology
Backend	Python 3.10+ with Flask framework
Database	MySQL (PyMySQL connector)
Frontend	HTML5, CSS3, Jinja2 templating, vanilla JavaScript
Package Manager	uv (Python package manager)
Deployment	Railway (cloud), localhost (development)

CHAPTER 2: PROBLEM STATEMENT & OBJECTIVES

2.1 Problem Statement

Traditional paper-based quiz systems in educational institutions suffer from several critical inefficiencies:

- Manual grading is time-consuming and error-prone
- Record-keeping across multiple assessments is cumbersome
- Students must wait days for results, delaying feedback
- No mechanism to prevent a student from viewing answers before all have submitted
- Paper-based systems offer no means to track performance trends over time

Existing online platforms often fail to address these issues adequately — they may lack role-based access control, result publication controls, or department-specific subject management.

2.2 Objectives

The primary objectives of this project are:

3. Develop a web-based quiz management system accessible on both desktop and mobile devices.
4. Implement role-based authentication with automatic role detection for teachers and students.
5. Enable teachers to create and manage quizzes, subjects, and questions (MCQ and True/False).
6. Implement automated grading with instant score calculation upon quiz submission.
7. Provide time-bound quiz functionality with automatic submission upon timer expiry.
8. Allow teachers to control result visibility — results remain hidden until explicitly published.
9. Maintain comprehensive records of quiz attempts and student performance in a relational database.
10. Implement enrollment-based access so students only see quizzes relevant to their enrolled subjects.
11. Randomize question order per attempt to reduce the risk of answer sharing between students.

CHAPTER 3: METHODOLOGY

3.1 System Architecture

The application follows a three-tier architecture:

- Presentation Layer: Jinja2 HTML templates rendered by Flask, styled with CSS, with JavaScript for the quiz timer.
- Application Layer: Python Flask with a Blueprint-based modular structure (auth, teacher, student blueprints).
- Data Layer: MySQL relational database accessed via PyMySQL with raw SQL queries.

3.2 Database Design

The database consists of 11 tables designed to capture the full lifecycle of quiz management:

Table	Purpose
Department	Stores the 8 engineering departments
Teacher	Teacher accounts with staff_id as login credential
Student	Student accounts with roll_number as login credential
Subject	Subjects created by teachers, linked to departments
Enrollment	Many-to-many mapping of students to subjects
Quiz	Quiz metadata including duration, marks, and result visibility
Question	MCQ and True/False questions linked to quizzes
Option	Answer options with correct answer flag
QuizAttempt	Tracks each student's attempt with score and timestamp
StudentAnswer	Records each answer submitted in an attempt

3.3 Key Design Decisions

- Single login portal: Role detection is performed by checking the Student table first (by roll number), then the Teacher table (by staff ID). This simplifies the login experience while maintaining clear role separation.
- Raw SQL queries: No ORM was used. All database interactions use raw SQL via PyMySQL, making the SQL logic transparent and assessable for a DBS lab context.
- Enrollment filter: Students only see quizzes for subjects they are enrolled in, implemented via a JOIN between Quiz, Subject, and Enrollment tables.
- Dual enforcement of attempt uniqueness: A UNIQUE KEY constraint on (quiz_id, student_id) in QuizAttempt enforces one attempt per student at the database level, with a Flask-level check providing a clean user-facing error.

- Frontend timer: The quiz countdown timer runs entirely in JavaScript in the browser, auto-submitting the form when it reaches zero. This keeps the backend stateless.
- Controlled result visibility: A results_published boolean flag on each quiz controls whether students can see their scores and answer breakdowns.

3.4 User Flows

Teacher flow:

Register (staff ID) → Login → Dashboard → Create Subject → Create Quiz → Add Questions → View Attempts → Publish Results

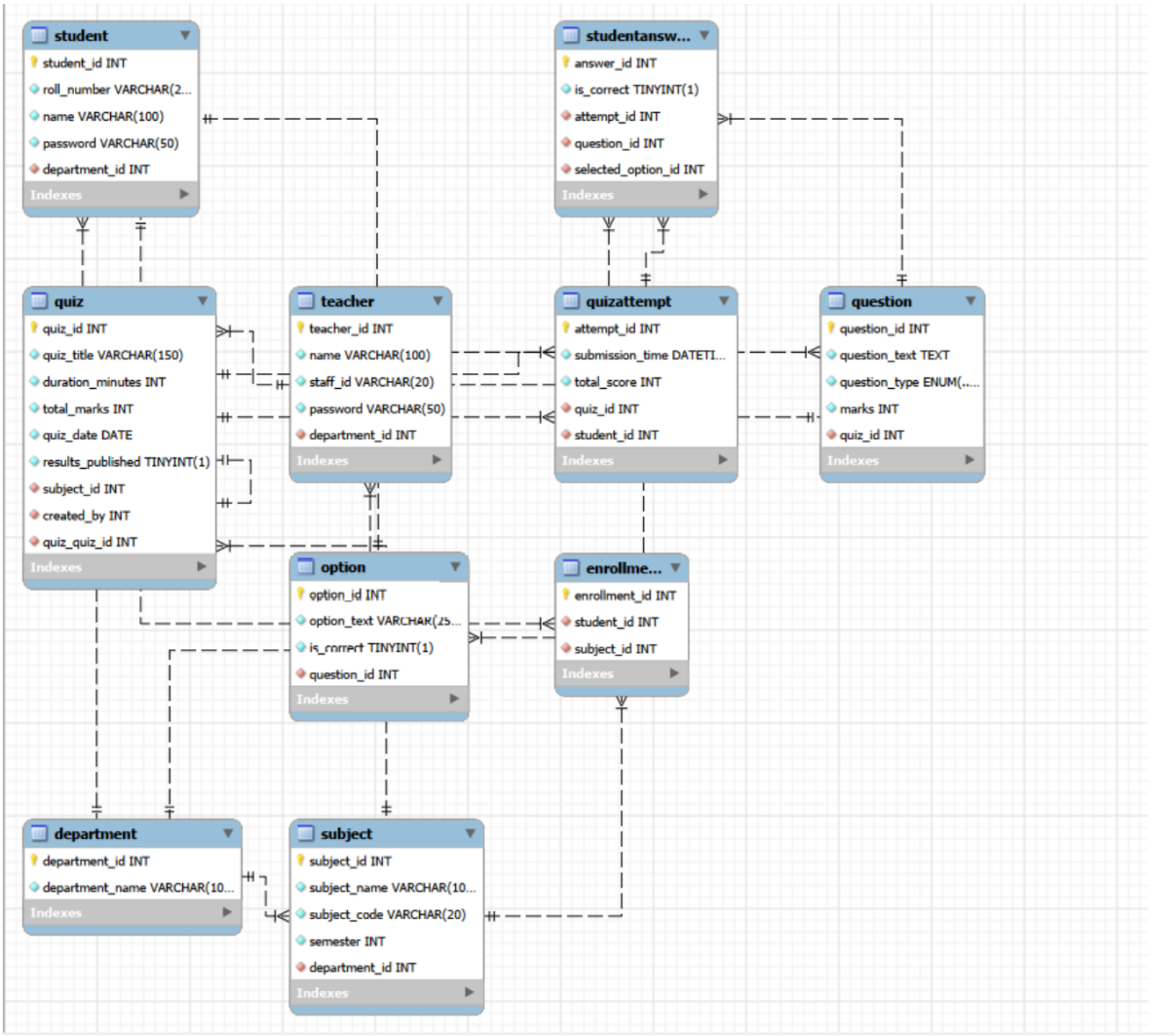
Student flow:

Register (roll number, enroll in subjects) → Login → Dashboard → Take Quiz (timer active) → Submit → View Result (after publication)

CHAPTER 4: ER DIAGRAM & RELATIONAL TABLES

4.1 Entity-Relationship Diagram

The ER diagram below illustrates all 11 entities, their attributes, and relationships. All relationships are One-to-Many (1:M) unless stated otherwise.



4.2 Entities and Attributes

- Department: department_id (PK), department_name
- Teacher: teacher_id (PK), name, staff_id (UNIQUE), password, department_id (FK)
- Student: student_id (PK), roll_number (UNIQUE), name, password, department_id (FK)
- Subject: subject_id (PK), subject_name, subject_code (UNIQUE), semester, department_id (FK)
- Enrollment: enrollment_id (PK), student_id (FK), subject_id (FK) — UNIQUE(student_id, subject_id)
- Quiz: quiz_id (PK), quiz_title, duration_minutes, total_marks, quiz_date, results_published, subject_id (FK), created_by (FK → Teacher)

- Question: question_id (PK), question_text, question_type (MCQ/TrueFalse), marks, quiz_id (FK)
- Option: option_id (PK), option_text, is_correct, question_id (FK)
- QuizAttempt: attempt_id (PK), submission_time, total_score, quiz_id (FK), student_id (FK) — UNIQUE(quiz_id, student_id)
- StudentAnswer: answer_id (PK), is_correct, attempt_id (FK), question_id (FK), selected_option_id (FK)

4.3 Relationships

Relationship	Cardinality	Description
Department → Teacher	1:M	One department has many teachers
Department → Student	1:M	One department has many students
Department → Subject	1:M	One department offers many subjects
Teacher → Quiz	1:M	One teacher creates many quizzes
Subject → Quiz	1:M	One subject has many quizzes
Student ↔ Subject	M:N	Via Enrollment table
Quiz → Question	1:M	One quiz has many questions
Question → Option	1:M	One question has 2 or 4 options
Student → QuizAttempt	1:M	One student makes many attempts (one per quiz)
QuizAttempt → StudentAnswer	1:M	One attempt has many answers

4.4 Sample Data

The schema is seeded with the following data for demonstration:

- 8 Departments including CCE, CSE, Mechanical, Electrical, Chemical, Industrial, Mechatronics, and CS with AIML
- 2 Teachers: Dr. Ramesh Kumar (CCE101) and Prof. Sunita Rao (CSE101)
- 4 Students: Ankitha (240953194), Arnav Varshney (240953382), Shreeniketh E R (240953148), Test Student (240953001)
- 1 Subject: Database Systems (CCE2212), Semester 4, CCE Department
- 1 Quiz: DBS Unit 1 Quiz — 15 minutes, 10 marks, 5 questions (3 MCQ + 2 True/False)
- All 4 seed students enrolled in Database Systems

CHAPTER 5: DDL COMMANDS & SQL QUERIES

5.1 Database and Table Creation (DDL)

The following DDL commands create the complete schema:

```
CREATE DATABASE quiz_db; USE quiz_db;
-- Department Table
CREATE TABLE Department (   department_id INT AUTO_INCREMENT PRIMARY KEY,   department_name VARCHAR(100) NOT NULL UNIQUE );
-- Teacher Table (staff_id as login credential)
CREATE TABLE Teacher (   teacher_id INT AUTO_INCREMENT PRIMARY KEY,   name VARCHAR(100) NOT NULL,   staff_id VARCHAR(20) NOT NULL UNIQUE,   password VARCHAR(50) NOT NULL,   department_id INT NOT NULL,   FOREIGN KEY (department_id) REFERENCES Department(department_id) );
-- Student Table (roll_number as login credential)
CREATE TABLE Student (   student_id INT AUTO_INCREMENT PRIMARY KEY,   roll_number VARCHAR(20) NOT NULL UNIQUE,   name VARCHAR(100) NOT NULL,   password VARCHAR(50) NOT NULL,   department_id INT NOT NULL,   FOREIGN KEY (department_id) REFERENCES Department(department_id) );
-- Enrollment Table (Student ↔ Subject M:N relationship)
CREATE TABLE Enrollment (   enrollment_id INT AUTO_INCREMENT PRIMARY KEY,   student_id INT NOT NULL,   subject_id INT NOT NULL,   FOREIGN KEY (student_id) REFERENCES Student(student_id) ON DELETE CASCADE,   FOREIGN KEY (subject_id) REFERENCES Subject(subject_id) ON DELETE CASCADE,   UNIQUE KEY unique_enrollment (student_id, subject_id) );
-- QuizAttempt Table (one attempt per student per quiz enforced at DB level)
CREATE TABLE QuizAttempt (   attempt_id INT AUTO_INCREMENT PRIMARY KEY,   submission_time DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,   total_score INT NOT NULL DEFAULT 0,   quiz_id INT NOT NULL,   student_id INT NOT NULL,   FOREIGN KEY (quiz_id) REFERENCES Quiz(quiz_id),   FOREIGN KEY (student_id) REFERENCES Student(student_id),   UNIQUE KEY unique_attempt (quiz_id, student_id) );
```

5.2 Key SQL Queries Used in the Application

Login — Role Detection:

```
SELECT student_id, name, password FROM Student WHERE roll_number = %s;
SELECT teacher_id, name, password FROM Teacher WHERE staff_id = %s;
```

Student Dashboard — Enrollment Filter:

```
SELECT q.quiz_id, q.quiz_title, q.duration_minutes, q.total_marks,   q.quiz_date, q.results_published, s.subject_name, s.subject_code,   qa.attempt_id, qa.total_score FROM Quiz q JOIN Subject s ON q.subject_id = s.subject_id JOIN Enrollment e ON e.subject_id = q.subject_id AND e.student_id = %s LEFT JOIN QuizAttempt qa ON q.quiz_id = qa.quiz_id AND qa.student_id = %s ORDER BY q.quiz_date DESC;
```

Automated Grading — Check Correct Answer:

```
SELECT is_correct FROM `Option` WHERE option_id = %s AND question_id = %s;
```

Result Breakdown (after result publication):

SELECT sa.is_correct, q.question_text, q.question_type, q.marks, sel.option_text AS selected_text, correct_opt.option_text AS correct_text FROM StudentAnswer sa JOIN Question q ON sa.question_id = q.question_id JOIN `Option` sel ON sa.selected_option_id = sel.option_id JOIN `Option` correct_opt ON correct_opt.question_id = q.question_id AND correct_opt.is_correct = TRUE WHERE sa.attempt_id = %s ORDER BY q.question_id;

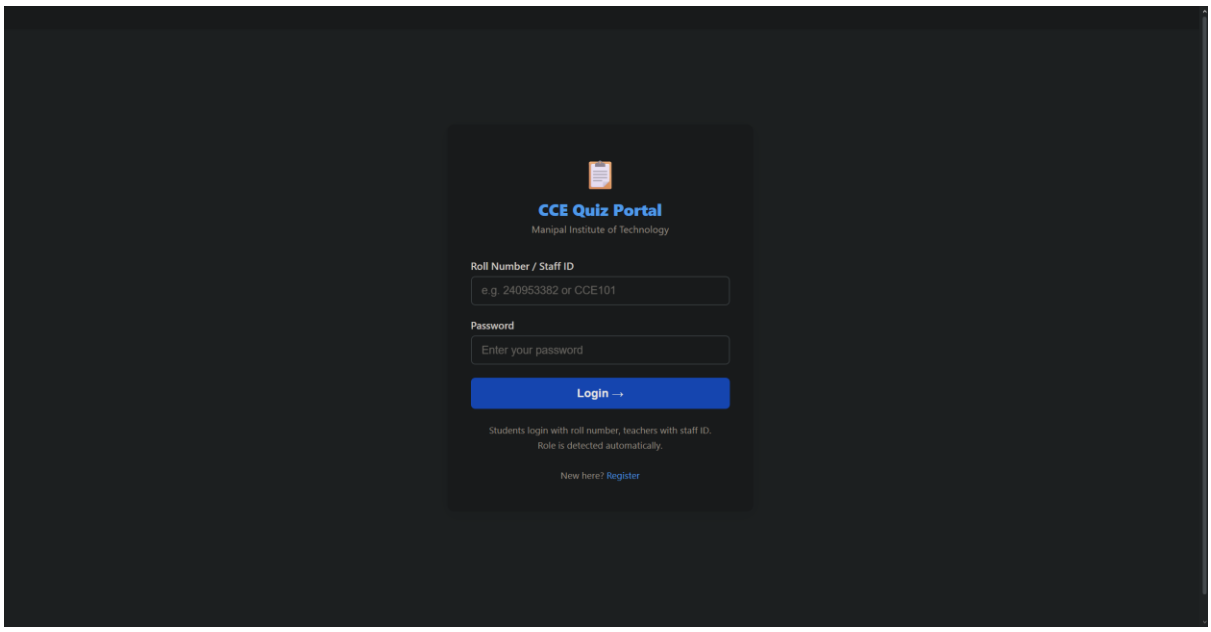
Teacher Results View (ranked by score):

SELECT st.name AS student_name, st.roll_number, qa.total_score, qa.submission_time FROM QuizAttempt qa JOIN Student st ON qa.student_id = st.student_id WHERE qa.quiz_id = %s ORDER BY qa.total_score DESC;

CHAPTER 6: RESULTS & SNAPSHOTS

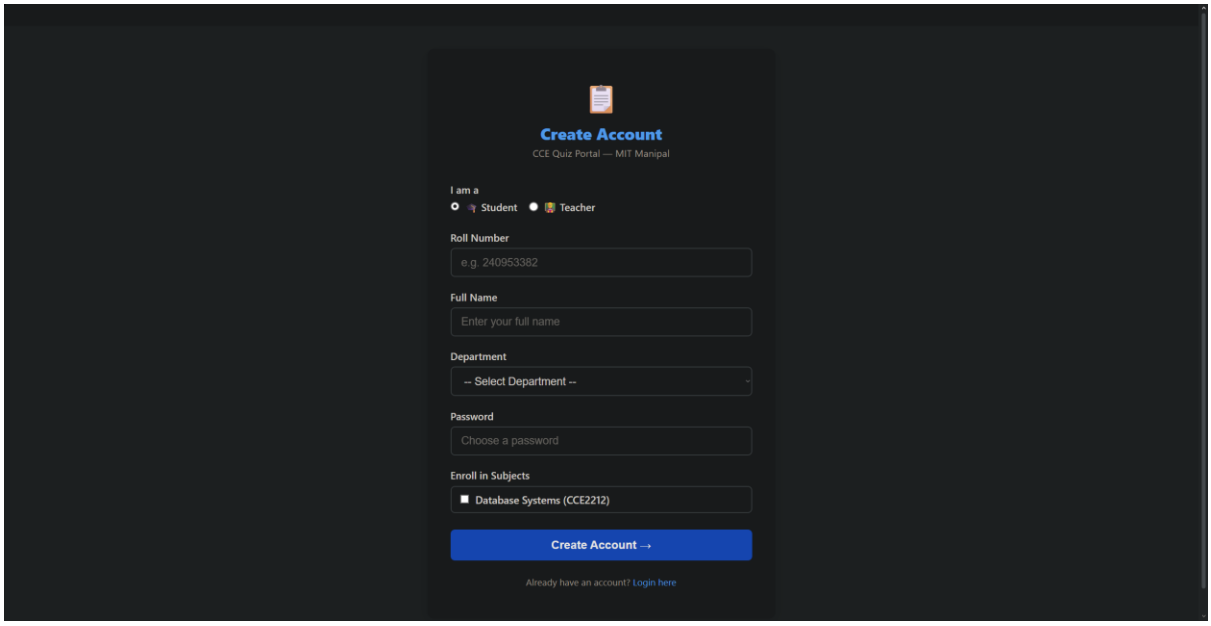
The following snapshots demonstrate the key screens of the College Quiz Management System. All features were tested and verified on both localhost and the Railway cloud deployment.

6.1 Login Page



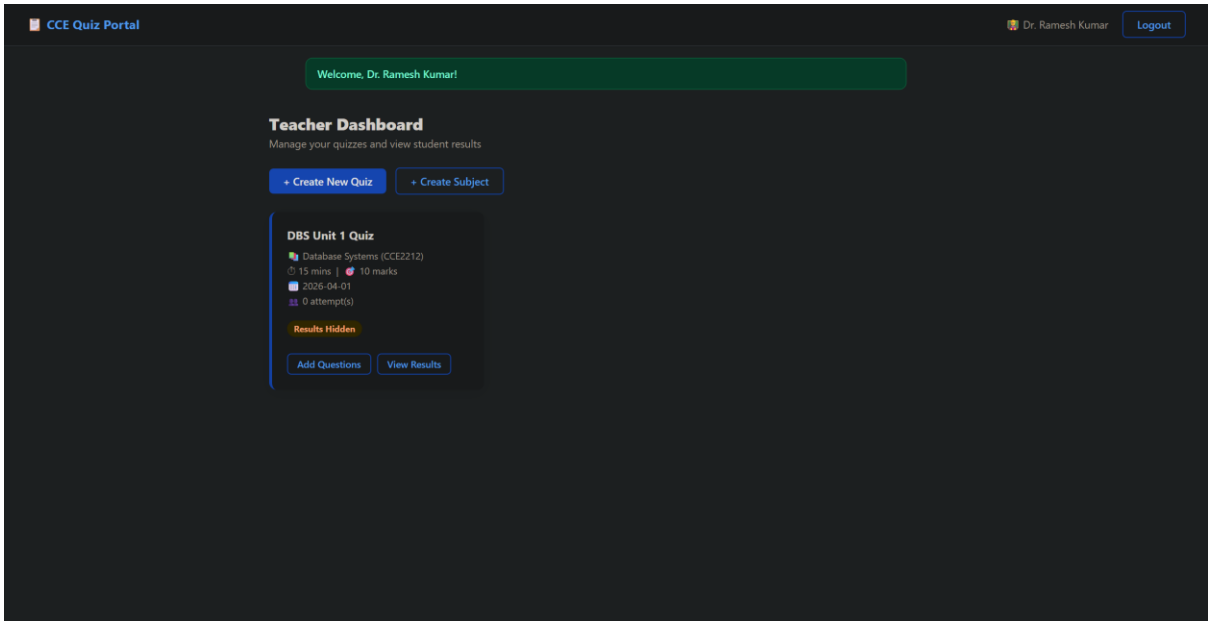
The single login portal accepts roll numbers for students and staff IDs for teachers. Role is detected automatically by querying both tables.

6.2 Registration Page



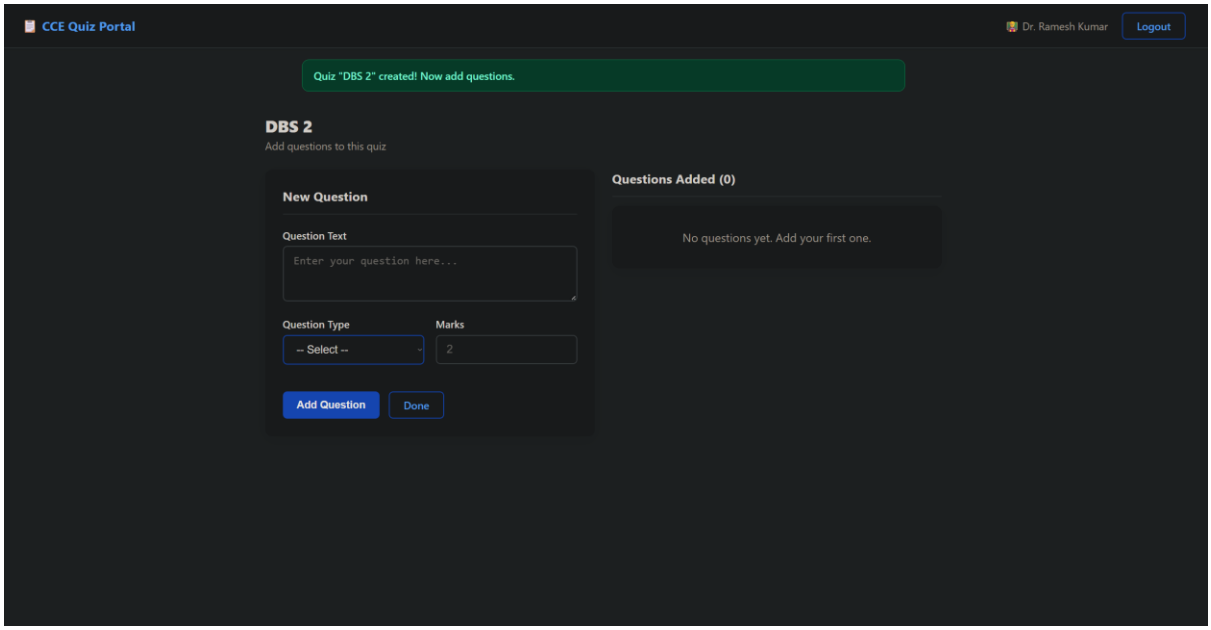
Students self-enroll in subjects at registration. Teacher registration captures staff ID and department.

6.3 Teacher Dashboard



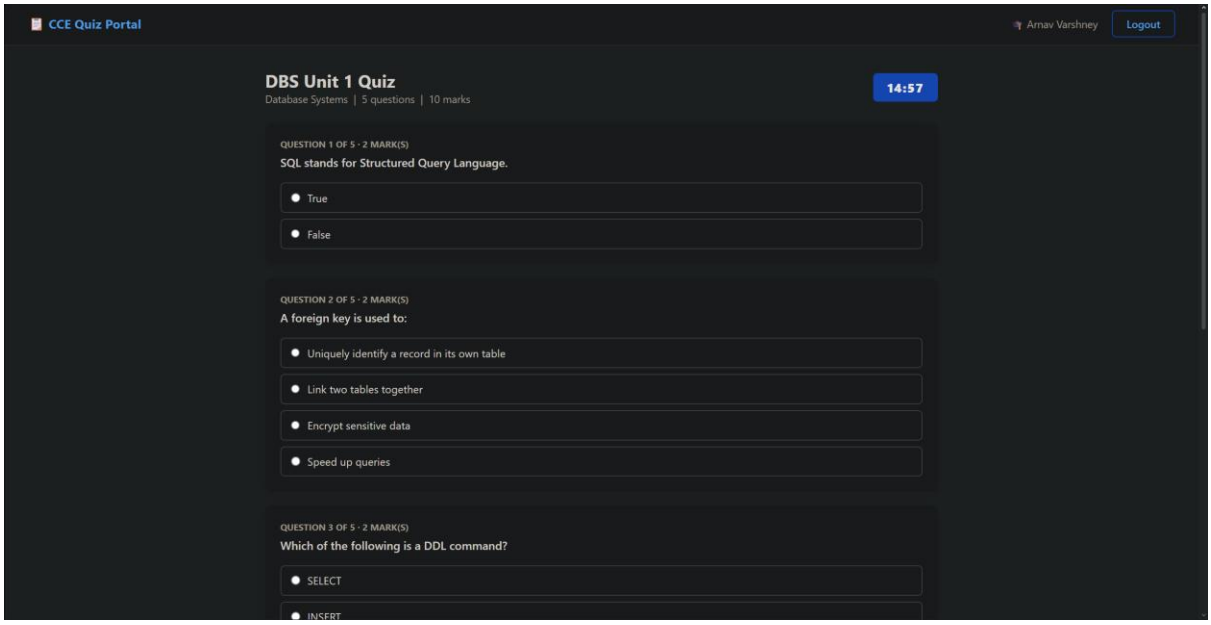
The teacher dashboard displays all quizzes created by the logged-in teacher with attempt counts and result publication status.

6.4 Add Question Page



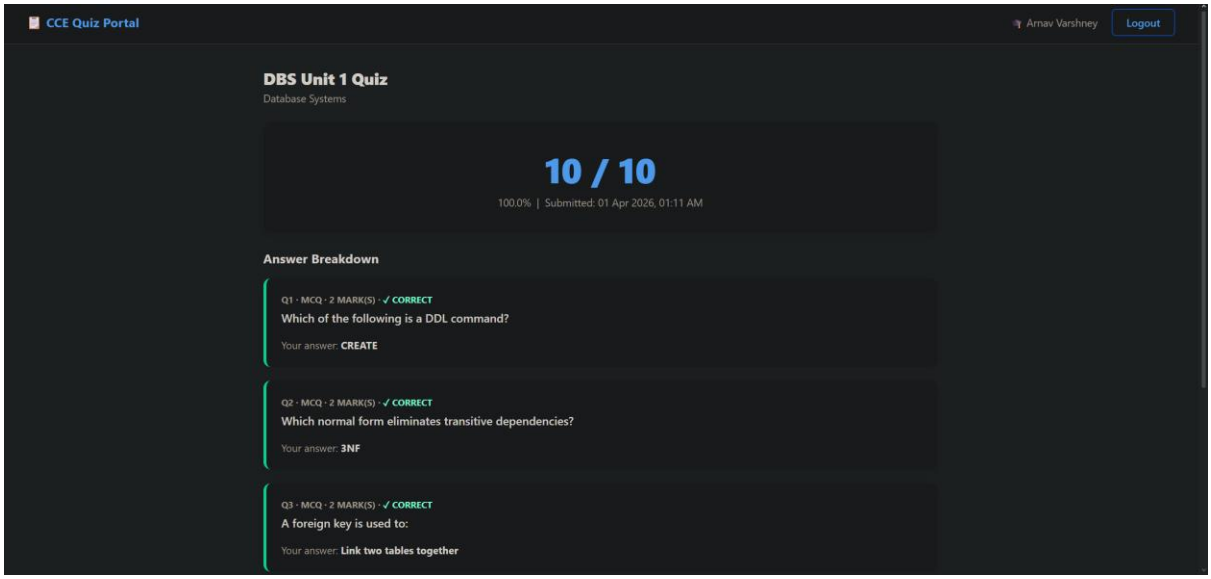
Teachers can add MCQ questions (4 options, radio button for correct answer) or True/False questions. Existing questions are listed on the right with a delete option.

6.5 Student Quiz Interface



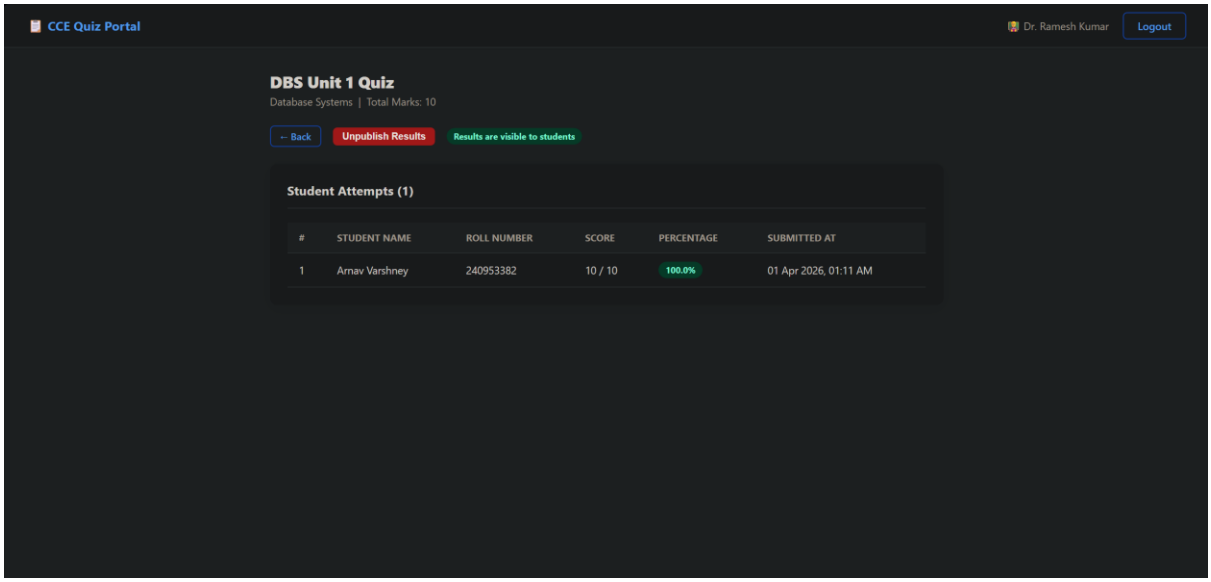
The quiz interface shows a countdown timer in the top right. Questions are rendered in randomized order. The form auto-submits when the timer reaches zero.

6.6 Result Page



After results are published by the teacher, students can view their total score and a per-question breakdown showing their selected answer and the correct answer.

6.7 Teacher Results View



Teachers can view all student attempts ranked by score, with percentage badges and submission timestamps.

CHAPTER 7: CONCLUSION, LIMITATIONS & FUTURE WORK

7.1 Conclusion

The College Quiz Management System successfully addresses the core inefficiencies of paper-based assessments in educational institutions. The system provides a secure, role-based web platform that automates the full quiz lifecycle — from creation to grading to result publication.

The project demonstrates the practical application of relational database design concepts including normalization, foreign key constraints, composite unique keys, and multi-table JOIN queries. The use of raw SQL throughout the application (with no ORM abstraction) makes the database interactions transparent and directly assessable.

The system was successfully deployed on Railway's cloud platform, making it accessible from any network without requiring students to connect to a local hotspot.

7.2 Limitations

- Passwords are stored as plain text. A production system would require bcrypt or similar hashing.
- The quiz timer runs on the frontend (JavaScript) only, making it theoretically bypassable by a technically savvy student. A backend timestamp check would be more secure.
- No email verification during registration — any student can register with any roll number.
- Only MCQ and True/False question types are supported. Short answer and descriptive questions are not implemented.
- No admin role exists — subject creation is restricted to teachers but there is no central administrator.
- The free Railway tier has limited uptime and compute resources, making it unsuitable for large concurrent user loads.

7.3 Future Work

- Implement password hashing using bcrypt for secure credential storage.
- Add server-side timer validation by recording quiz start time in the database and rejecting submissions past the deadline.
- Introduce an admin role for managing departments, subjects, and user accounts centrally.
- Add support for short-answer and descriptive question types with manual grading by teachers.
- Implement performance analytics dashboards for teachers showing score distributions and question difficulty metrics.
- Add email notifications for result publication using Flask-Mail.
- Extend the enrollment system to allow teachers to approve or reject student enrollment requests.

CHAPTER 8: REFERENCES

12. Flask Documentation — https://flask.palletsprojects.com/
13. PyMySQL Documentation — https://pymysql.readthedocs.io/
14. MySQL 8.0 Reference Manual — https://dev.mysql.com/doc/refman/8.0/en/
15. Jinja2 Template Designer Documentation — https://jinja.palletsprojects.com/
16. Railway Deployment Documentation — https://docs.railway.app/
17. Ramez Elmasri and Shamkant B. Navathe, Fundamentals of Database Systems, 7th Edition, Pearson, 2015.
18. Abraham Silberschatz, Henry F. Korth, and S. Sudarshan, Database System Concepts, 7th Edition, McGraw-Hill, 2019.